

A. Cora 실습 보드 (개요)

2021년 8월 20일 금요일 오전 10:11

(1) 실습에는 Zynq 개발 보드는 다음 중 하나를 사용한다.

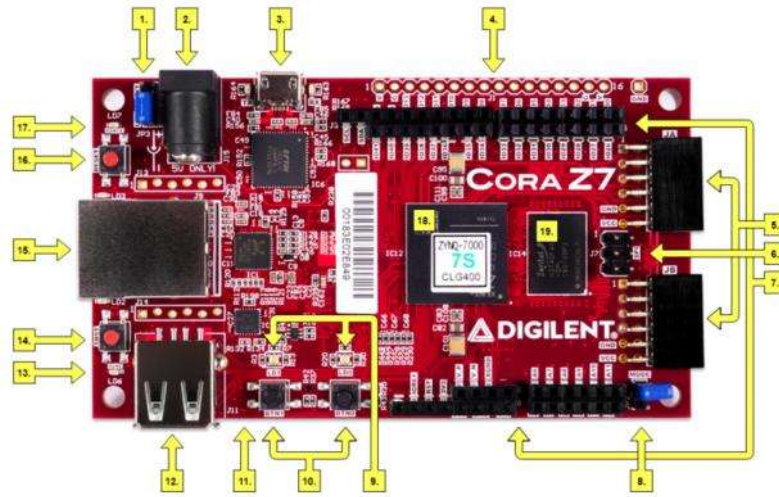
- Cora Z7-07S (single core)
- Cora Z7-10 (dual core)
- Zybo Z7 (dual core)

(1) 모든 실습 보드는 Zynq7000 SoC를 장착하고 있으며, 보드에서 사용 할 수 있는 I/O (LED, Push button, Switch, GPIO port 등) 만 다르기 때문에 기본적인 설계 실습은 동일 하게 진행 할 수 있으며, I/O의 사용 및 Pin assignment 만 보드에 맞게 설정 해주면 된다.

- 단, 프로젝트 생성할 때, **해당 board file**을 선택해야 한다.
- Cora Z7-07S과 Cora Z7-10은 동일한 보드이며 Port 도 동일하다. 따라서 모든 실습은 동일하게 진행할 수 있다.
- Zybo-Z7의 경우 설계는 동일하며 단, I/O planning (핀할당) 만 보드에 맞도록 설정해 주면 된다.

(1) Cora Z7-07S board

- ZYNQ Processor
 - 667MHz single-core Cortex-A9 processor
 - 3600 Programmable logic slices (FPGA)
 - 60 DSP slices
 - 225 KB of block RAM
- 1G Ethernet, USB 2.0, SDIO
- Low-bandwidth peripheral controllers: SPI, UART, CAN, I2C
- 1 MSPS internal analog-digital converter



Callout	Description	Callout	Description
1	Power select jumper (Ext. supply / USB)	11	microSD card slot (underside of board)
2	Power jack (for optional ext. supply)	12	USB host port
3	Shared USB JTAG / UART port	13	FPGA programming DONE LED
4	Unloaded expansion header	14	Processor subsystem reset button
5	Pmod connectors	15	Ethernet port
6	SPI header (Arduino/ChipKIT compatible)	16	Power on reset button
7	Arduino/ChipKIT shield connectors	17	Power good LED
8	Programming mode jumper (JTAG / microSD)	18	Zynq-7000
9	User tri-color LEDs	19	DDR3L memory
10	User push buttons		

(2) Cora Z7-10 board : Dual core Cortex-A9 processor

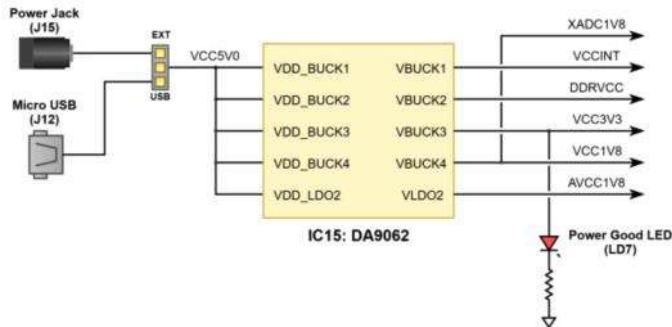
a. Cora Z7 개발보드 주요 기능

2021년 8월 20일 금요일 오전 10:29

Cora Z7 개발 보드는 Xilinx의 Vivado design tool WebPACK (free version)을 이용하여 FPGA 설계를 할 수 있으며, 같이 제공되는 Xilinx SDK는 Zynq PS processor의 응용 프로그램을 개발할 수 있는 환경을 제공한다.

(1) Functional Description (기능 설명)

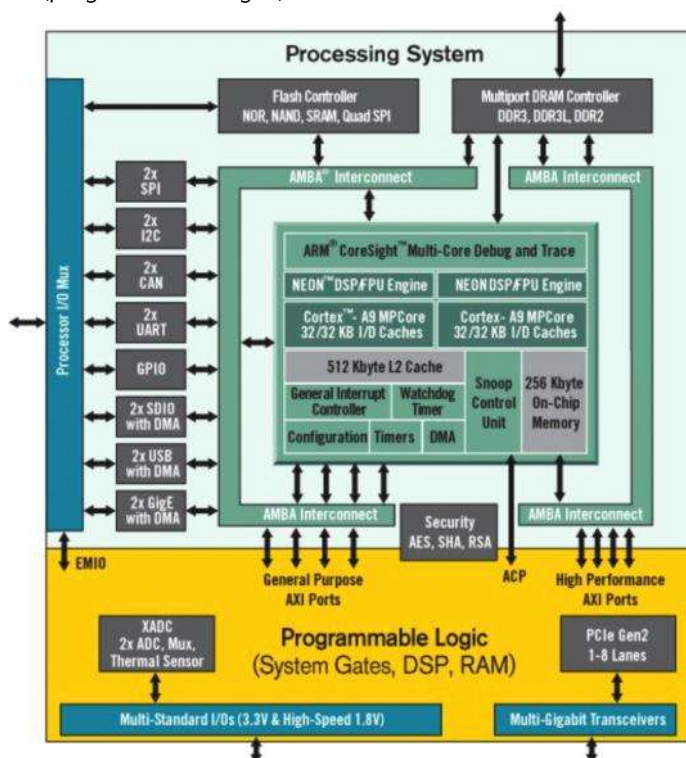
- Power supply
 - Cora Z7은 5V의 전원을 필요로 한다.
 - USB-JTAG (J12) connector를 통하여 공급
 - 또는, Power Jack(J15)을 통하여 5V power supply로 공급 (최소 1A 이상)



- Regulator (IC 15 : DA9062)에서 1.8V, 3.3V 등 다음과 같이 공급한다.

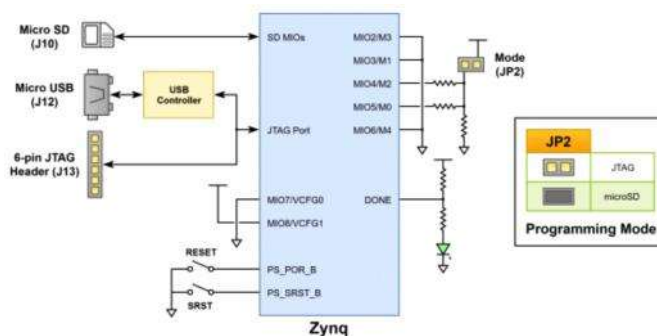
Supply	Circuits	Device	Maximum Current
5.0V	Onboard Regulators, Arduino/chipKit Shield Connector, RGB LEDs	IC4: ON Semiconductor NCP380 ¹⁾	
3.3V	FPGA I/O, USB port, Ethernet	IC15: Dialog Semiconductor DA9062	2A
1.0V	FPGA, Ethernet	IC15: Dialog Semiconductor DA9062	2.5A
1.8V	FPGA Auxiliary, USB port, Ethernet	IC15: Dialog Semiconductor DA9062	1.5A
1.35V	FPGA, DDR3L memory	IC15: Dialog Semiconductor DA9062	2.5A
1.8V	FPGA XADC	IC15: Dialog Semiconductor DA9062	100mA

- Zynq SoC의 구조
 - PS(Processing System) : APU(Cortex-A9), DDR3 memory controller, Peripheral controller , 54 Multiplexed IO pin , EMIO
 - PL (programmable Logic) : FPGA



(2) Zynq configuration 과정

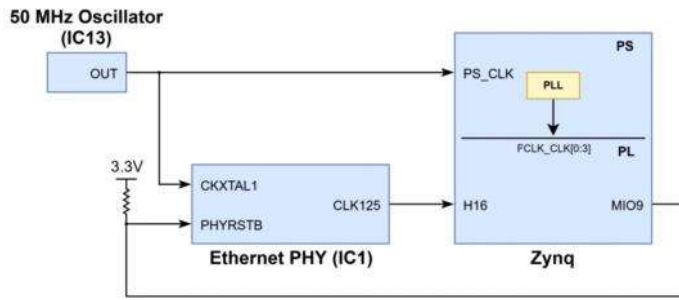
- Zynq의 configuration 과정은 FPGA device의 configuration (programming)과는 조금 다르다. 즉, Zynq SoC은 Processor가 master로서 FPGA의 programming, on-chip peripheral의 configuration의 기능을 수행한다. 따라서 3단계 과정으로 configuration이 진행된다.
 - Stage 0
 - 전원이 인가되면, Zynq reset이 되며 processor는 **BootROM의 코드가 실행**이 된다.
 - BootROM은 개발자가 생성하여 SD card또는 EEPROM에 저장해둔 **FSBL**을 on-chip RAM으로 불러온다.
 - Stage 1
 - FSBL이 실행되면서 **PC component의 configuration** (DDR memory 등),
 - FPGA programming을 위한 bitstream이 있는 경우, **FPGA의 configuration**도 수행한다.
 - 그리고, **user application이 load** 된다.
 - Stage 2
 - FSBL이 load한 user application이 PS에서 실행이 된다.
- Zynq configuration을 위한 Boot Image는 Xilinx SDK에서 생성을 할 수 있으며, SD card 또는 EEPROM과 같은 non-volatile memory에 저장하여, 전원이 인가되면 자동으로 configuration이 되도록 할 수 있다.
- Cora Z7의 configuration 방법



- **microSD Boot mode** (JP2를 jumper로 연결)
 - J10의 microSD에 Boot Image가 저장되어 있는 경우
 - microSD card는 FAT32로 format한다.
 - Xilinx tool로 생성한 Zynq Boot Image를 SD card에 복사하고 BOOT.bin으로 변경한다.
 - Cora Z7의 J10에 microSD card를 넣는다.
 - JP2는 microSD mode로 선택한다.
 - 전원을 인가하면 board는 microSD에서 자동으로 booting 된다.
- **JTAG Boot mode**
 - Xilinx tool에 의해서 software가 loading 된다.
 - 또한 JTAG를 통하여 PL을 직접 configure할 수 있다.

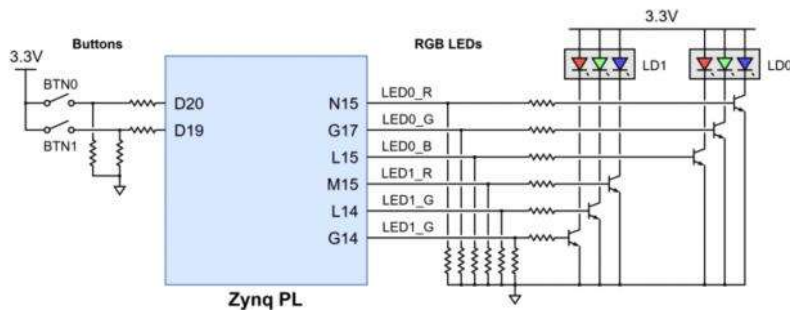
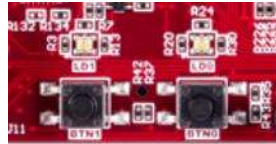
(3) Clock sources

- Processing System의 clock : 50MHz clock이 Zynq PS clock (PS_CLK)으로 공급된다.
- PL clock : 125MHz의 clock이 H16 port로 직접 공급 된다. 이 클럭은 PPL또는 MMCM에 입력이 되어, 다양한 주파수의 클럭을 다시 생성 할 수 있다. 단, PHYRSTB (MIO9) 가 low이면 125MHz clock은 disable 된다.



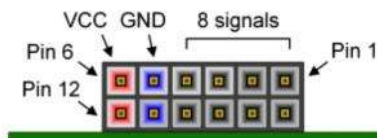
(6) Cora Z7 board의 Basic I/O

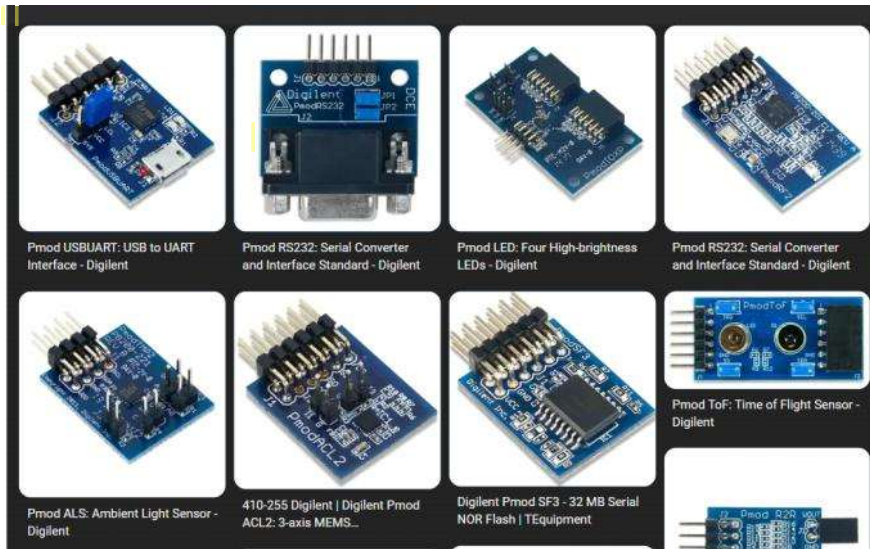
- 입력 버튼 2개를 사용 가능하다.
 - BTN0 : D20 port (push일 때 'High')
 - BTN1 : D19 port (push일 때 'High')
- RGB LED 2개 사용 가능하다.
 - RGB led는 각각 3개의 color를 출력할 수 있는 단자를 가지고 있다.
 - 각각의 입력은 50% 이하의 PWM으로 구동하기를 권장한다.
 - 또한 R,G,B 의 독립적인 PWM 구동으로 다양한 색상을 만들 수 있다.



(7) PMOD connector

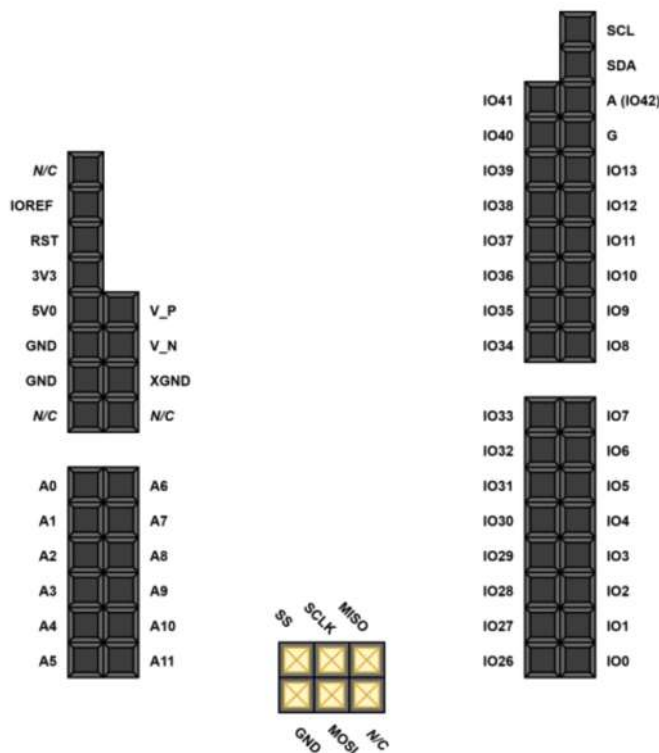
- Digilent standard 2x6 connector : JA 와 JB
- 3.3v, Gnd 그리고 8개의 signal pin으로 구성된다.





(8) Arduino Shield Connector

- Cora Z7은 standard Arduino shield 를 장착 할 수 있는 Pin을 가지고 있다.
- 이 Shield connector는 대부분의 Arduino shield와 호환이 된다.
- 45개의 pin은 FPGA의 GPIO에 연결이 되어 있다. FPGA는 digital read/write, UART, I2C, SPI , PWM 등 필요한 기능을 유연하게 설계하여 연결 할 수 있다.
- AN0 ~ AN5은 0~3.3V 범위의 single-ended analog input으로 사용 한다.
- AN6 ~ AN11은 3 pair로 0~1V 범위의 differential analog input으로 사용할 수 있다.
- 단, Cora Z7 shield의 입력은 3.3V logic이므로, 3.4V 이상의 입력을 인가할 경우 손상을 입을 수 있으니 5V logic을 연결하지 않도록 주의해야 한다.



T15
T14
V13
U14

Pin Name	Shield Function	Cora Z7 Connection
I00-I013, IO26-IO41, A (IO42)	General purpose I/O pins	See Section titled "Shield Digital I/O"
SCL	I2C Clock	See Section titled "Shield Digital I/O"
SDA	I2C Data	See Section titled "Shield Digital I/O"
SCK	SPI Clock	See Section titled "Shield Digital I/O"
SS	SPI Slave Select	See Section titled "Shield Digital I/O"
MOSI	SPI Master-Out Slave-In	See Section titled "Shield Digital I/O"

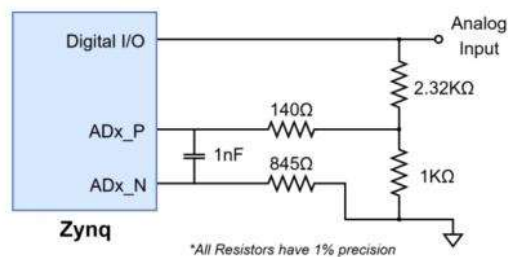
MISO	SPI Master-In Slave-In	See Section titled "Shield Digital I/O"
A0-A5	Single-Ended Analog Input	See Section titled "Shield Analog I/O"
A6-A11	Differential Analog Input	See Section titled "Shield Analog I/O"
V_P, V_N	Dedicated Differential Analog Input	See Section titled "Shield Analog I/O"
XGND	XADC Analog Ground	Connected to net used to drive the XADC ground reference on the FPGA (VREFN)
N/C	Not Connected	Not Connected
IOREF	Digital I/O Voltage reference	Connected to the Cora Z7 3.3V Power Rail (See the "Power Supplies" section)
RST	Reset to Shield	Connected to the red "SRST" button and a Digital I/O of the FPGA. When JP1 is shorted, it is also connected to the DTR signal of the FTDI USB-UART bridge.
3V3	3.3V Power Rail	Connected to the Cora Z7 3.3V Power Rail (See the "Power Supplies" section)
5V0	5.0V Power Rail	Connected to the Cora Z7 5.0V Power Rail (See the "Power Supplies" section)
GND, G	Ground	Connected to the Ground plane of Cora Z7

○ Shield connector의 Digital I/O 규격

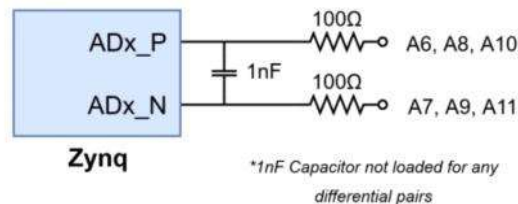
- FPGA IO pin 보호를 위하여 200ohm의 직렬 저항이 연결 되어 있다.
- Digital IO pin의 maximum rating 전압은 -0.4V 이상 ~ 3.75V 이하이다.

○ Shield connector의 Analog I/O 규격

- A0~A11 및 V_P,V_N은 Zynq의 내장 XADC 입력으로 사용된다.
- Single ended Analog input A0~A5 ADC 입력회로는 다음과 같다. XADC의 입력 범위는 0~1V 이므로 3.3V의 singel ended analog 입력을 받아들이기 위하여 다음과 같이 전압 분배회로가 적용이 되어 있다.



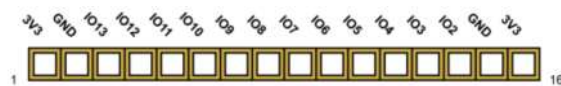
- Differential Analog input A6~A11은 0~1V의 전압범위를 가지며, 입력회로는 다음과 같다.

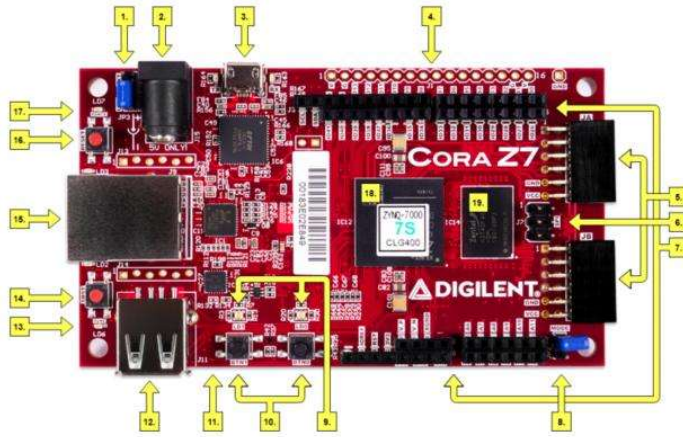


- V_P/V_N은 XADC의 AD0_P/ADx_N에 직접 연결이 된 differential analog input이며, 0~1V의 범위를 가진다.

(9) Unloaded Expansion Header

- 추가적으로 12개의 digital I/O가 J1 connector에 연결이 되어 있다.
- 총 16pin의 header connector에는 양쪽에 3.3V와 Gnd가 각각 연결이 되어 있다.





Callout	Description	Callout	Description
1	Power select jumper (Ext. supply / USB)	11	microSD card slot (underside of board)
2	Power jack (for optional ext. supply)	12	USB host port
3	Shared USB JTAG / UART port	13	FPGA programming DONE LED
4	Unloaded expansion header	14	Processor subsystem reset button
5	Pmod connectors	15	Ethernet port
6	SPI header (Arduino/ChipKIT compatible)	16	Power on reset button
7	Arduino/ChipKIT shield connectors	17	Power good LED
8	Programming mode jumper (JTAG / microSD)	18	Zynq-7000
9	User tri-color LEDs	19	DDR3L memory
10	User push buttons		

b. Cora Z7 보드의 I/O 정의 (XDC file)



CoraZ7.xdc

2021년 8월 21일 토요일 오전 10:31

(1) Cora Z7 board의 I/O pin assignment를 위한 XDC file

- Board의 모든 I/O port 에 대한 I/O 전압과 port number를 정의하고 있다.
- 필요한 I/O assignment 부분은 comment 문자('#')을 지우고 사용하면 된다.
- XDC file을 사용하는 방법은 Vivado 실습의 pin 할당 과정에서 다시 설명하도록 한다.

```
## This file is a general .xdc for the Cora Z7-07S Rev. B
## To use it in a project:
## - uncomment the lines corresponding to used pins
## - rename the used ports (in each line, after get_ports) according to the top level signal names in the project

## PL System Clock
#set_property -dict { PACKAGE_PIN H16 IOSTANDARD LVCMOS33 } [get_ports { clk }]; #IO_L13P_T2_MRCC_35 Sch=sysclk
#create_clock -add -name sys_clk_pin -period 8.00 -waveform {0 4} [get_ports { clk }];#set

## RGB LEDs
#set_property -dict { PACKAGE_PIN L15 IOSTANDARD LVCMOS33 } [get_ports { led0_b }]; #IO_L22N_T3_AD7N_35 Sch=led0_b
#set_property -dict { PACKAGE_PIN G17 IOSTANDARD LVCMOS33 } [get_ports { led0_g }]; #IO_L16P_T2_35 Sch=led0_g
#set_property -dict { PACKAGE_PIN N15 IOSTANDARD LVCMOS33 } [get_ports { led0_r }]; #IO_L21P_T3_DQS_AD14P_35 Sch=led0_r
#set_property -dict { PACKAGE_PIN G14 IOSTANDARD LVCMOS33 } [get_ports { led1_b }]; #IO_0_35 Sch=led1_b
#set_property -dict { PACKAGE_PIN L14 IOSTANDARD LVCMOS33 } [get_ports { led1_g }]; #IO_L22P_T3_AD7P_35 Sch=led1_g
#set_property -dict { PACKAGE_PIN M15 IOSTANDARD LVCMOS33 } [get_ports { led1_r }]; #IO_L23N_T3_35 Sch=led1_r

## Buttons
#set_property -dict { PACKAGE_PIN D20 IOSTANDARD LVCMOS33 } [get_ports { btn[0] }]; #IO_L4N_T0_35 Sch=btn[0]
#set_property -dict { PACKAGE_PIN D19 IOSTANDARD LVCMOS33 } [get_ports { btn[1] }]; #IO_L4P_T0_35 Sch=btn[1]

## Pmod Header JA
#set_property -dict { PACKAGE_PIN Y18 IOSTANDARD LVCMOS33 } [get_ports { ja[0] }]; #IO_L17P_T2_34 Sch=ja_p[1]
#set_property -dict { PACKAGE_PIN Y19 IOSTANDARD LVCMOS33 } [get_ports { ja[1] }]; #IO_L17N_T2_34 Sch=ja_n[1]
#set_property -dict { PACKAGE_PIN Y16 IOSTANDARD LVCMOS33 } [get_ports { ja[2] }]; #IO_L7P_T1_34 Sch=ja_p[2]
#set_property -dict { PACKAGE_PIN Y17 IOSTANDARD LVCMOS33 } [get_ports { ja[3] }]; #IO_L7N_T1_34 Sch=ja_n[2]
#set_property -dict { PACKAGE_PIN T10 IOSTANDARD LVCMOS33 } [get_ports { ja[4] }]; #IO_L12P_T1_MRCC_34 Sch=ja_p[3]
#set_property -dict { PACKAGE_PIN U19 IOSTANDARD LVCMOS33 } [get_ports { ja[5] }]; #IO_L12N_T1_MRCC_34 Sch=ja_n[3]
#set_property -dict { PACKAGE_PIN W18 IOSTANDARD LVCMOS33 } [get_ports { ja[6] }]; #IO_L22P_T3_34 Sch=ja_p[4]
#set_property -dict { PACKAGE_PIN W19 IOSTANDARD LVCMOS33 } [get_ports { ja[7] }]; #IO_L22N_T3_34 Sch=ja_n[4]

## Pmod Header JB
#set_property -dict { PACKAGE_PIN W14 IOSTANDARD LVCMOS33 } [get_ports { jb[0] }]; #IO_L8P_T1_34 Sch=jb_p[1]
#set_property -dict { PACKAGE_PIN Y14 IOSTANDARD LVCMOS33 } [get_ports { jb[1] }]; #IO_L8N_T1_34 Sch=jb_n[1]
#set_property -dict { PACKAGE_PIN T11 IOSTANDARD LVCMOS33 } [get_ports { jb[2] }]; #IO_L1P_T0_34 Sch=jb_p[2]
#set_property -dict { PACKAGE_PIN T10 IOSTANDARD LVCMOS33 } [get_ports { jb[3] }]; #IO_L1N_T0_34 Sch=jb_n[2]
#set_property -dict { PACKAGE_PIN V16 IOSTANDARD LVCMOS33 } [get_ports { jb[4] }]; #IO_L18P_T2_34 Sch=jb_p[3]
#set_property -dict { PACKAGE_PIN W16 IOSTANDARD LVCMOS33 } [get_ports { jb[5] }]; #IO_L18N_T2_34 Sch=jb_n[3]
#set_property -dict { PACKAGE_PIN V12 IOSTANDARD LVCMOS33 } [get_ports { jb[6] }]; #IO_L4P_T0_34 Sch=jb_p[4]
#set_property -dict { PACKAGE_PIN W13 IOSTANDARD LVCMOS33 } [get_ports { jb[7] }]; #IO_L4N_T0_34 Sch=jb_n[4]

## Crypto SDA
#set_property -dict { PACKAGE_PIN J15 IOSTANDARD LVCMOS33 } [get_ports { crypto_sda }];

## Dedicated Analog Inputs
#set_property -dict { PACKAGE_PIN K9 IOSTANDARD LVCMOS33 } [get_ports { v_p }]; #VP_0 Sch=xadc_v_p
#set_property -dict { PACKAGE_PIN L10 IOSTANDARD LVCMOS33 } [get_ports { v_n }]; #VN_0 Sch=xadc_v_n

## ChipKit Outer Analog Header - as Single-Ended Analog Inputs
## NOTE: These ports can be used as single-ended analog inputs with voltages from 0-3.3V (ChipKit analog pins A0-A5) or as digital I/O.
## WARNING: Do not use both sets of constraints at the same time!
#set_property -dict { PACKAGE_PIN E17 IOSTANDARD LVCMOS33 } [get_ports { vaux1_p }]; #IO_L3P_T0_DQS_AD1P_35 Sch=ck_an_p[0]
#set_property -dict { PACKAGE_PIN D18 IOSTANDARD LVCMOS33 } [get_ports { vaux1_n }]; #IO_L3N_T0_DQS_AD1N_35 Sch=ck_an_n[0]
#set_property -dict { PACKAGE_PIN E18 IOSTANDARD LVCMOS33 } [get_ports { vaux9_p }]; #IO_L5P_T0_AD9P_35 Sch=ck_an_p[1]
#set_property -dict { PACKAGE_PIN E19 IOSTANDARD LVCMOS33 } [get_ports { vaux9_n }]; #IO_L5N_T0_AD9N_35 Sch=ck_an_n[1]
#set_property -dict { PACKAGE_PIN K14 IOSTANDARD LVCMOS33 } [get_ports { vaux6_p }]; #IO_L20P_T3_AD6P_35 Sch=ck_an_p[2]
#set_property -dict { PACKAGE_PIN J14 IOSTANDARD LVCMOS33 } [get_ports { vaux6_n }]; #IO_L20N_T3_AD6N_35 Sch=ck_an_n[2]
#set_property -dict { PACKAGE_PIN K16 IOSTANDARD LVCMOS33 } [get_ports { vaux15_p }]; #IO_L24P_T3_AD15P_35 Sch=ck_an_p[3]
#set_property -dict { PACKAGE_PIN J16 IOSTANDARD LVCMOS33 } [get_ports { vaux15_n }]; #IO_L24N_T3_AD15N_35 Sch=ck_an_n[3]
#set_property -dict { PACKAGE_PIN J20 IOSTANDARD LVCMOS33 } [get_ports { vaux5_p }]; #IO_L17P_T2_AD5P_35 Sch=ck_an_p[4]
#set_property -dict { PACKAGE_PIN H20 IOSTANDARD LVCMOS33 } [get_ports { vaux5_n }]; #IO_L17N_T2_AD5N_35 Sch=ck_an_n[4]
#set_property -dict { PACKAGE_PIN G19 IOSTANDARD LVCMOS33 } [get_ports { vaux13_p }]; #IO_L18P_T2_AD13P_35 Sch=ck_an_p[5]
#set_property -dict { PACKAGE_PIN G20 IOSTANDARD LVCMOS33 } [get_ports { vaux13_n }]; #IO_L18N_T2_AD13N_35 Sch=ck_an_n[5]
## ChipKit Outer Analog Header - as Digital I/O
## NOTE: The following constraints should be used when using these ports as digital I/O.
#set_property -dict { PACKAGE_PIN F17 IOSTANDARD LVCMOS33 } [get_ports { ck_a0 }]; #IO_L6N_T0_VREF_35 Sch=ck_a[0]
#set_property -dict { PACKAGE_PIN J19 IOSTANDARD LVCMOS33 } [get_ports { ck_a1 }]; #IO_L10N_T1_AD11N_35 Sch=ck_a[1]
#set_property -dict { PACKAGE_PIN K17 IOSTANDARD LVCMOS33 } [get_ports { ck_a2 }]; #IO_L12P_T1_MRCC_35 Sch=ck_a[2]
#set_property -dict { PACKAGE_PIN L16 IOSTANDARD LVCMOS33 } [get_ports { ck_a3 }]; #IO_L11P_T1_SRCC_35 Sch=ck_a[3]
#set_property -dict { PACKAGE_PIN N16 IOSTANDARD LVCMOS33 } [get_ports { ck_a4 }]; #IO_L21N_T3_DQS_AD14N_35 Sch=ck_a[4]
#set_property -dict { PACKAGE_PIN P14 IOSTANDARD LVCMOS33 } [get_ports { ck_a5 }]; #IO_L6P_T0_34 Sch=ck_a[5]

## ChipKit Inner Analog Header - as Differential Analog Inputs
## NOTE: These ports can be used as differential analog inputs with voltages from 0-1.0V (ChipKit analog pins A6-A11) or as digital I/O.
## WARNING: Do not use both sets of constraints at the same time!
#set_property -dict { PACKAGE_PIN C20 IOSTANDARD LVCMOS33 } [get_ports { vaux0_p }]; #IO_L1P_T0_AD0P_35 Sch=ad_p[0]
#set_property -dict { PACKAGE_PIN B20 IOSTANDARD LVCMOS33 } [get_ports { vaux0_n }]; #IO_L1N_T0_AD0N_35 Sch=ad_n[0]
#set_property -dict { PACKAGE_PIN F19 IOSTANDARD LVCMOS33 } [get_ports { vaux12_p }]; #IO_L15P_T2_DQS_AD12P_35 Sch=ad_p[12]
#set_property -dict { PACKAGE_PIN F20 IOSTANDARD LVCMOS33 } [get_ports { vaux12_n }]; #IO_L15N_T2_DQS_AD12N_35 Sch=ad_n[12]
#set_property -dict { PACKAGE_PIN B19 IOSTANDARD LVCMOS33 } [get_ports { vaux8_p }]; #IO_L2P_T0_AD8P_35 Sch=ad_p[8]
#set_property -dict { PACKAGE_PIN A20 IOSTANDARD LVCMOS33 } [get_ports { vaux8_n }]; #IO_L2N_T0_AD8N_35 Sch=ad_n[8]
## ChipKit Inner Analog Header - as Digital I/O
## NOTE: The following constraints should be used when using the inner analog header ports as digital I/O.
#set_property -dict { PACKAGE_PIN C20 IOSTANDARD LVCMOS33 } [get_ports { ck_a6 }]; #IO_L1P_T0_AD0P_35 Sch=ad_p[0]
#set_property -dict { PACKAGE_PIN B20 IOSTANDARD LVCMOS33 } [get_ports { ck_a7 }]; #IO_L1N_T0_AD0N_35 Sch=ad_n[0]
#set_property -dict { PACKAGE_PIN F19 IOSTANDARD LVCMOS33 } [get_ports { ck_a8 }]; #IO_L15P_T2_DQS_AD12P_35 Sch=ad_p[12]
#set_property -dict { PACKAGE_PIN F20 IOSTANDARD LVCMOS33 } [get_ports { ck_a9 }]; #IO_L15N_T2_DQS_AD12N_35 Sch=ad_n[12]
#set_property -dict { PACKAGE_PIN B19 IOSTANDARD LVCMOS33 } [get_ports { ck_a10 }]; #IO_L2P_T0_AD8P_35 Sch=ad_p[8]
#set_property -dict { PACKAGE_PIN A20 IOSTANDARD LVCMOS33 } [get_ports { ck_a11 }]; #IO_L2N_T0_AD8N_35 Sch=ad_n[8]

## ChipKit Outer Digital Header
#set_property -dict { PACKAGE_PIN U14 IOSTANDARD LVCMOS33 } [get_ports { ck_io0 }]; #IO_L11P_T1_SRCC_34 Sch=ck_io[0]
```

```

#set_property -dict { PACKAGE_PIN V13 IOSTANDARD LVCMOS33 } [get_ports { ck_io1 }]; #IO_L3N_T0_DQS_34 Sch=ck_io[1]
#set_property -dict { PACKAGE_PIN T14 IOSTANDARD LVCMOS33 } [get_ports { ck_io2 }]; #IO_L5P_T0_34 Sch=ck_io[2]
#set_property -dict { PACKAGE_PIN T15 IOSTANDARD LVCMOS33 } [get_ports { ck_io3 }]; #IO_L5N_T0_34 Sch=ck_io[3]
#set_property -dict { PACKAGE_PIN V17 IOSTANDARD LVCMOS33 } [get_ports { ck_io4 }]; #IO_L21P_T3_DQS_34 Sch=ck_io[4]
#set_property -dict { PACKAGE_PIN V18 IOSTANDARD LVCMOS33 } [get_ports { ck_io5 }]; #IO_L21N_T3_DQS_34 Sch=ck_io[5]
#set_property -dict { PACKAGE_PIN R17 IOSTANDARD LVCMOS33 } [get_ports { ck_io6 }]; #IO_L19N_T3_VREF_34 Sch=ck_io[6]
#set_property -dict { PACKAGE_PIN R14 IOSTANDARD LVCMOS33 } [get_ports { ck_io7 }]; #IO_L6N_T0_VREF_34 Sch=ck_io[7]
#set_property -dict { PACKAGE_PIN N18 IOSTANDARD LVCMOS33 } [get_ports { ck_io8 }]; #IO_L13P_T2_MRCC_34 Sch=ck_io[8]
#set_property -dict { PACKAGE_PIN M18 IOSTANDARD LVCMOS33 } [get_ports { ck_io9 }]; #IO_L8N_T1_AD10N_35 Sch=ck_io[9]
#set_property -dict { PACKAGE_PIN U15 IOSTANDARD LVCMOS33 } [get_ports { ck_io10 }]; #IO_L11N_T1_SRCC_34 Sch=ck_io[10]
#set_property -dict { PACKAGE_PIN R18 IOSTANDARD LVCMOS33 } [get_ports { ck_io11 }]; #IO_L12N_T1_MRCC_35 Sch=ck_io[11]
#set_property -dict { PACKAGE_PIN J18 IOSTANDARD LVCMOS33 } [get_ports { ck_io12 }]; #IO_L14P_T2_AD4P_SRCC_35 Sch=ck_io[12]
#set_property -dict { PACKAGE_PIN G15 IOSTANDARD LVCMOS33 } [get_ports { ck_io13 }]; #IO_L19N_T3_VREF_35 Sch=ck_io[13]

## ChipKit Inner Digital Header
#set_property -dict { PACKAGE_PIN R16 IOSTANDARD LVCMOS33 } [get_ports { ck_io26 }]; #IO_L19P_T3_34 Sch=ck_io[26]
#set_property -dict { PACKAGE_PIN U12 IOSTANDARD LVCMOS33 } [get_ports { ck_io27 }]; #IO_L2N_T0_34 Sch=ck_io[27]
#set_property -dict { PACKAGE_PIN U13 IOSTANDARD LVCMOS33 } [get_ports { ck_io28 }]; #IO_L3P_T0_DQS_PUDC_B_34 Sch=ck_io[28]
#set_property -dict { PACKAGE_PIN V15 IOSTANDARD LVCMOS33 } [get_ports { ck_io29 }]; #IO_L10P_T1_34 Sch=ck_io[29]
#set_property -dict { PACKAGE_PIN T16 IOSTANDARD LVCMOS33 } [get_ports { ck_io30 }]; #IO_L9P_T1_DQS_34 Sch=ck_io[30]
#set_property -dict { PACKAGE_PIN U17 IOSTANDARD LVCMOS33 } [get_ports { ck_io31 }]; #IO_L9N_T1_DQS_34 Sch=ck_io[31]
#set_property -dict { PACKAGE_PIN T17 IOSTANDARD LVCMOS33 } [get_ports { ck_io32 }]; #IO_L20P_T3_34 Sch=ck_io[32]
#set_property -dict { PACKAGE_PIN R18 IOSTANDARD LVCMOS33 } [get_ports { ck_io33 }]; #IO_L20N_T3_34 Sch=ck_io[33]
#set_property -dict { PACKAGE_PIN P18 IOSTANDARD LVCMOS33 } [get_ports { ck_io34 }]; #IO_L23N_T3_34 Sch=ck_io[34]
#set_property -dict { PACKAGE_PIN N17 IOSTANDARD LVCMOS33 } [get_ports { ck_io35 }]; #IO_L23P_T3_34 Sch=ck_io[35]
#set_property -dict { PACKAGE_PIN M17 IOSTANDARD LVCMOS33 } [get_ports { ck_io36 }]; #IO_L8P_T1_AD10P_35 Sch=ck_io[36]
#set_property -dict { PACKAGE_PIN L17 IOSTANDARD LVCMOS33 } [get_ports { ck_io37 }]; #IO_L11N_T1_SRCC_35 Sch=ck_io[37]
#set_property -dict { PACKAGE_PIN H17 IOSTANDARD LVCMOS33 } [get_ports { ck_io38 }]; #IO_L13N_T2_MRCC_35 Sch=ck_io[38]
#set_property -dict { PACKAGE_PIN H18 IOSTANDARD LVCMOS33 } [get_ports { ck_io39 }]; #IO_L14N_T2_AD4N_SRCC_35 Sch=ck_io[39]
#set_property -dict { PACKAGE_PIN G18 IOSTANDARD LVCMOS33 } [get_ports { ck_io40 }]; #IO_L16N_T2_35 Sch=ck_io[40]
#set_property -dict { PACKAGE_PIN L20 IOSTANDARD LVCMOS33 } [get_ports { ck_io41 }]; #IO_L9N_T1_DQS_AD3N_35 Sch=ck_io[41]

## ChipKit SPI
#set_property -dict { PACKAGE_PIN W15 IOSTANDARD LVCMOS33 } [get_ports { ck_miso }]; #IO_L10N_T1_34 Sch=ck_miso
#set_property -dict { PACKAGE_PIN T12 IOSTANDARD LVCMOS33 } [get_ports { ck_mosi }]; #IO_L2P_T0_34 Sch=ck_mosi
#set_property -dict { PACKAGE_PIN H15 IOSTANDARD LVCMOS33 } [get_ports { ck_sck }]; #IO_L19P_T3_35 Sch=ck_sck
#set_property -dict { PACKAGE_PIN F16 IOSTANDARD LVCMOS33 } [get_ports { ck_ss }]; #IO_L6P_T0_35 Sch=ck_ss

## ChipKit I2C
#set_property -dict { PACKAGE_PIN P16 IOSTANDARD LVCMOS33 } [get_ports { ck_scl }]; #IO_L24N_T3_34 Sch=ck_scl
#set_property -dict { PACKAGE_PIN P15 IOSTANDARD LVCMOS33 } [get_ports { ck_sda }]; #IO_L24P_T3_34 Sch=ck_sda

##Misc. ChipKit signals
#set_property -dict { PACKAGE_PIN M20 IOSTANDARD LVCMOS33 } [get_ports { ck_iaa }]; #IO_L7N_T1_AD2N_35 Sch=ck_iaa

## User Digital I/O Header J1
#set_property -dict { PACKAGE_PIN L19 IOSTANDARD LVCMOS33 } [get_ports { user_dio[1] }]; #IO_L9P_T1_DQS_AD3P_35 Sch=user_dio[1]
#set_property -dict { PACKAGE_PIN M19 IOSTANDARD LVCMOS33 } [get_ports { user_dio[2] }]; #IO_L7P_T1_AD2P_35 Sch=user_dio[2]
#set_property -dict { PACKAGE_PIN N20 IOSTANDARD LVCMOS33 } [get_ports { user_dio[3] }]; #IO_L14P_T2_SRCC_34 Sch=user_dio[3]
#set_property -dict { PACKAGE_PIN P20 IOSTANDARD LVCMOS33 } [get_ports { user_dio[4] }]; #IO_L14N_T2_SRCC_34 Sch=user_dio[4]
#set_property -dict { PACKAGE_PIN P19 IOSTANDARD LVCMOS33 } [get_ports { user_dio[5] }]; #IO_L13N_T2_MRCC_34 Sch=user_dio[5]
#set_property -dict { PACKAGE_PIN R19 IOSTANDARD LVCMOS33 } [get_ports { user_dio[6] }]; #IO_0_34 Sch=user_dio[6]
#set_property -dict { PACKAGE_PIN T20 IOSTANDARD LVCMOS33 } [get_ports { user_dio[7] }]; #IO_L15P_T2_DQS_34 Sch=user_dio[7]
#set_property -dict { PACKAGE_PIN T19 IOSTANDARD LVCMOS33 } [get_ports { user_dio[8] }]; #IO_25_34 Sch=user_dio[8]
#set_property -dict { PACKAGE_PIN U20 IOSTANDARD LVCMOS33 } [get_ports { user_dio[9] }]; #IO_L15N_T2_DQS_34 Sch=user_dio[9]
#set_property -dict { PACKAGE_PIN V20 IOSTANDARD LVCMOS33 } [get_ports { user_dio[10] }]; #IO_L16P_T2_34 Sch=user_dio[10]
#set_property -dict { PACKAGE_PIN W20 IOSTANDARD LVCMOS33 } [get_ports { user_dio[11] }]; #IO_L16N_T2_34 Sch=user_dio[11]
#set_property -dict { PACKAGE_PIN K19 IOSTANDARD LVCMOS33 } [get_ports { user_dio[12] }]; #IO_L10P_T1_AD11P_35 Sch=user_dio[12]

```

c. Cora Z7 보드 사용을 위한 준비

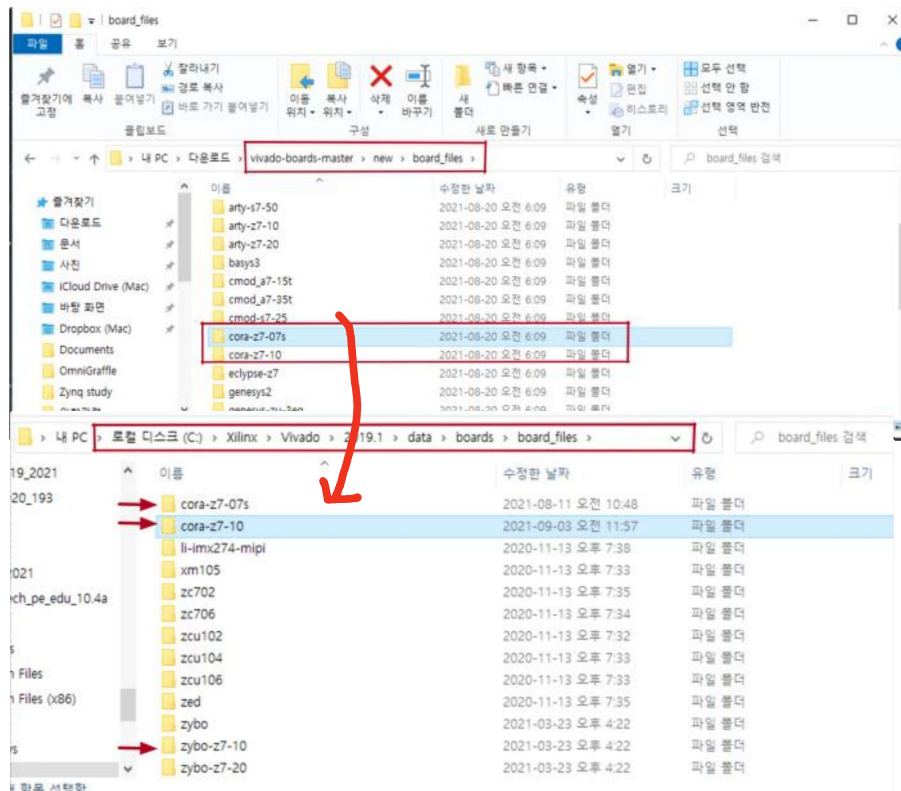
(1) Cora Z7 board file 설치

- Vivado에서 CoraZ7 board를 사용하기 위하여, Cora board file들을 Xilinx > **Vivado** > **2019.1** > **data** > **boards** > **board_files** 디렉토리에 복사한다.

- Board file을 아래 링크에서 다운로드한다.

<https://github.com/Digilent/vivado-boards/archive/master.zip?ga=2.225377682.929207999.1629512259-897421608.1629512259>

- 다운로드한 보드파일 **vivado-boards-master**에서 각자 사용할 보드에 해당하는 directory를 복사하면 되지만, Cora-z7-07s, Cora-z7-10 및 Zybo-10 보드 파일을 모두 선택하여 아래와 같이 복사를 해준다.
 - vivado-boards-master/new/board_files/cora0z7-07s directory를 Vivado의 board directory에 복사한다.
 - vivado-boards-master/new/board_files/cora0z7-10 directory를 Vivado의 board directory에 복사한다.
 - vivado-boards-master/new/board_files/zybo-z7-10 directory를 Vivado의 board directory에 복사한다.



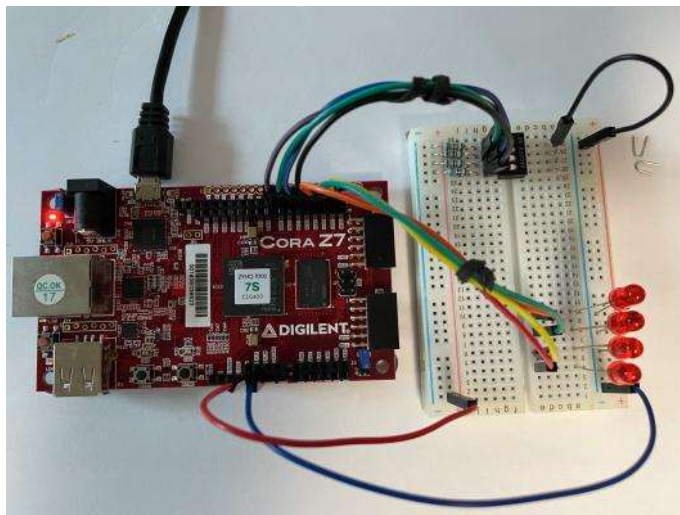
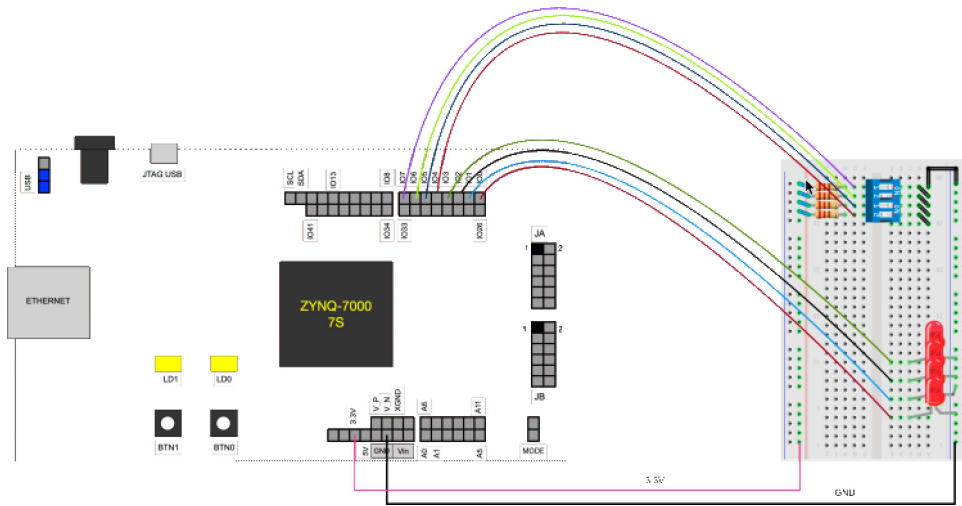
- (2) VIVADO는 JTAG를 이용하여 Cora 보드와 통신을 하므로, JTAG mode로 설정한다.

- (3) USB 전원을 사용하도록 Power Set-up을 USB mode로 설정한다.



(4) 실습을 위한 I/O 장치 확장

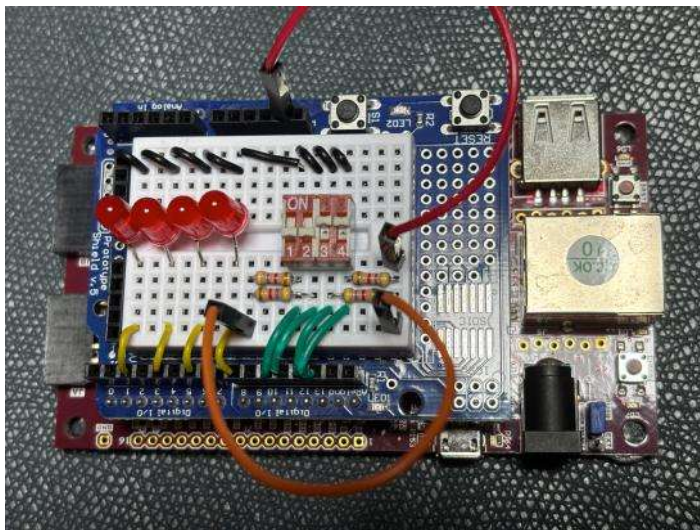
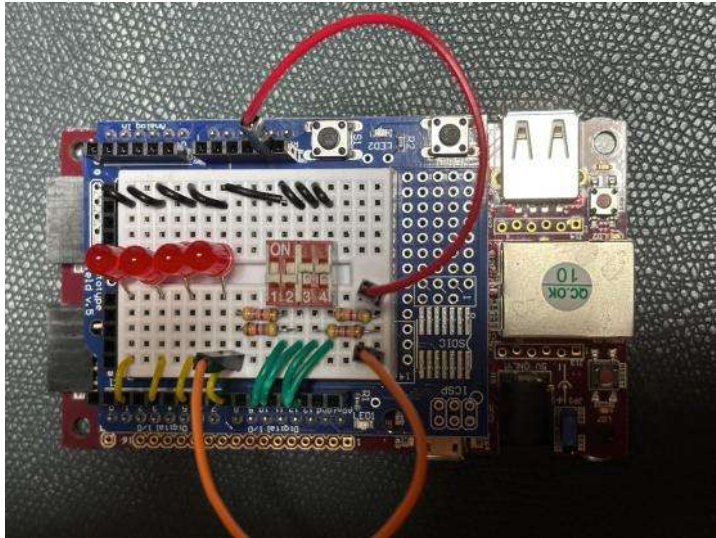
- Cora Z7 보드에 기본 실습을 위한 Switch 입력과 LED를 Breadboard를 이용하여 확장시킨다.
- 4bit LED 출력
 - IO0 ~ IO3 port를 LED의 Anode(+ 단자)에 연결한다.
 - LED의 Kathode (- 단자)는 전부 Ground에 연결한다.
 - 일반적으로 길이가 긴 단자가 Anode이지만, 직접 테스트 해보는 것이 필요하다. 수백 ohm 정도의 저항을 직렬로 연결하여 3~5V 전압에 연결해보면 극성을 확인할 수 있다.
- 4bit Switch 입력
 - IO4~IO7 port를 4bit DIP switch에 아래 회로와 같이 연결한다.
 - DIP switch의 아래쪽 4단자(on 표시)는 모두 Ground에 연결한다.
 - DIP switch의 위쪽은 수Kohm 정도의 저항을 이용하여 3.3V에 pullup 시킨다.
 - Pullup은 단자와 전원전압을 저항으로 연결 하는 것이다.
 - 따라서 이 단자가 switch open 되었을 경우 전원전압으로 high 상태가 되며, switch on 1과 short가 되면서 low 상태가 된다.
 - 아래 회로에서는 DIP switch 를 "on"으로 내리면 low, 올리면 high 상태를 IO4~IO7에 입력



d. Cora board with shield board

2025년 9월 30일 화요일 오전 10:08

- Sw[3:0] : 13,12,11,10
- Led[3:0] : 8,6,4,2
- Gnd를 연결해준다



- xdc file

PL System Clock

```
#set_property -dict { PACKAGE_PIN H16  IOSTANDARD LVCMOS33 } [get_ports { clk_125M }]; #IO_L13P_T2_MRCC_35 Sch=sysclk  
#create_clock -add -name sys_clk_pin -period 8.00 -waveform {0 4} [get_ports { clk_125M }];#set
```

RGB LEDs

```
#set_property -dict { PACKAGE_PIN L15  IOSTANDARD LVCMOS33 } [get_ports { led0_b }]; #IO_L22N_T3_AD7N_35 Sch=led0_b  
#set_property -dict { PACKAGE_PIN G17  IOSTANDARD LVCMOS33 } [get_ports { led0_g }]; #IO_L16P_T2_35 Sch=led0_g  
#set_property -dict { PACKAGE_PIN N15  IOSTANDARD LVCMOS33 } [get_ports { led0_r }]; #IO_L21P_T3_DQS_AD14P_35 Sch=led0_r  
#set_property -dict { PACKAGE_PIN G14  IOSTANDARD LVCMOS33 } [get_ports { led1_b }]; #IO_0_35 Sch=led1_b  
#set_property -dict { PACKAGE_PIN L14  IOSTANDARD LVCMOS33 } [get_ports { led1_g }]; #IO_L22P_T3_AD7P_35 Sch=led1_g  
#set_property -dict { PACKAGE_PIN M15  IOSTANDARD LVCMOS33 } [get_ports { led1_r }]; #IO_L23N_T3_35 Sch=led1_r
```

Buttons

```
#set_property -dict { PACKAGE_PIN D20 IOSTANDARD LVCMOS33 } [get_ports { btn0 }]; #IO_L4N_T0_35 Sch=btn[0]
#set_property -dict { PACKAGE_PIN D19 IOSTANDARD LVCMOS33 } [get_ports { btn[1] }]; #IO_L4P_T0_35 Sch=btn[1]
```

ChipKit Outer Digital Header

```
#set_property -dict { PACKAGE_PIN U14 IOSTANDARD LVCMOS33 } [get_ports { ck_io0 }]; #IO_L11P_T1_SRCC_34 Sch=ck_io[0]
#set_property -dict { PACKAGE_PIN V13 IOSTANDARD LVCMOS33 } [get_ports { ck_io1 }]; #IO_L3N_T0_DQS_34 Sch=ck_io[1]
set_property -dict { PACKAGE_PIN T14 IOSTANDARD LVCMOS33 } [get_ports { led4_tri_o[0] }]; #IO_L5P_T0_34 Sch=ck_io[2]
#set_property -dict { PACKAGE_PIN T15 IOSTANDARD LVCMOS33 } [get_ports { ck_io3 }]; #IO_L5N_T0_34 Sch=ck_io[3]
set_property -dict { PACKAGE_PIN V17 IOSTANDARD LVCMOS33 } [get_ports { led4_tri_o[1] }]; #IO_L21P_T3_DQS_34 Sch=ck_io[4]
#set_property -dict { PACKAGE_PIN V18 IOSTANDARD LVCMOS33 } [get_ports { ck_io5 }]; #IO_L21N_T3_DQS_34 Sch=ck_io[5]
set_property -dict { PACKAGE_PIN R17 IOSTANDARD LVCMOS33 } [get_ports { led4_tri_o[2] }]; #IO_L19N_T3_VREF_34 Sch=ck_io[6]
#set_property -dict { PACKAGE_PIN R14 IOSTANDARD LVCMOS33 } [get_ports { ck_io7 }]; #IO_L6N_T0_VREF_34 Sch=ck_io[7]
set_property -dict { PACKAGE_PIN N18 IOSTANDARD LVCMOS33 } [get_ports { led4_tri_o[3] }]; #IO_L13P_T2_MRCC_34 Sch=ck_io[8]
#set_property -dict { PACKAGE_PIN M18 IOSTANDARD LVCMOS33 } [get_ports { ck_io9 }]; #IO_L8N_T1_AD10N_35 Sch=ck_io[9]
set_property -dict { PACKAGE_PIN U15 IOSTANDARD LVCMOS33 } [get_ports { sw4_tri_o[0] }]; #IO_L11N_T1_SRCC_34 Sch=ck_io[10]
set_property -dict { PACKAGE_PIN K18 IOSTANDARD LVCMOS33 } [get_ports { sw4_tri_o[1] }]; #IO_L12N_T1_MRCC_35 Sch=ck_io[11]
set_property -dict { PACKAGE_PIN J18 IOSTANDARD LVCMOS33 } [get_ports { sw4_tri_o[2] }]; #IO_L14P_T2_AD4P_SRCC_35 Sch=ck_io[12]
set_property -dict { PACKAGE_PIN G15 IOSTANDARD LVCMOS33 } [get_ports { sw4_tri_o[3] }]; #IO_L19N_T3_VREF_35 Sch=ck_io[13]
```

B. ZYBO Z7 실습 보드 (개요)

2021년 7월 3일 토요일 오전 10:44

a. Zybo baord (개요)

2022년 11월 18일 금요일 오후 8:38

- Board 개요



The Zybo Z7.

ZYNQ Processor

- 667 MHz dual-core Cortex-A9 processor
- DDR3L memory controller with 8 DMA channels and 4 High Performance AXI3 Slave ports
- High-bandwidth peripheral controllers: 1G Ethernet, USB 2.0, SDIO
- Low-bandwidth peripheral controllers: SPI, UART, CAN, I2C
- Programmable from JTAG, Quad-SPI flash, and microSD card
- Programmable logic equivalent to Artix-7 FPGA

Memory

- 1 GB DDR3L with 32-bit bus @ 1066 MHz
- 16 MB Quad-SPI Flash with factory programmed 128-bit random number and 48-bit globally unique EUI-48/64™ compatible identifier

Power

- Powered from USB or any 5V external power source

USB and Ethernet

- Gigabit Ethernet PHY
- USB-JTAG Programming circuitry
- USB-UART bridge
- USB 2.0 OTG PHY with host and device support

Audio and Video

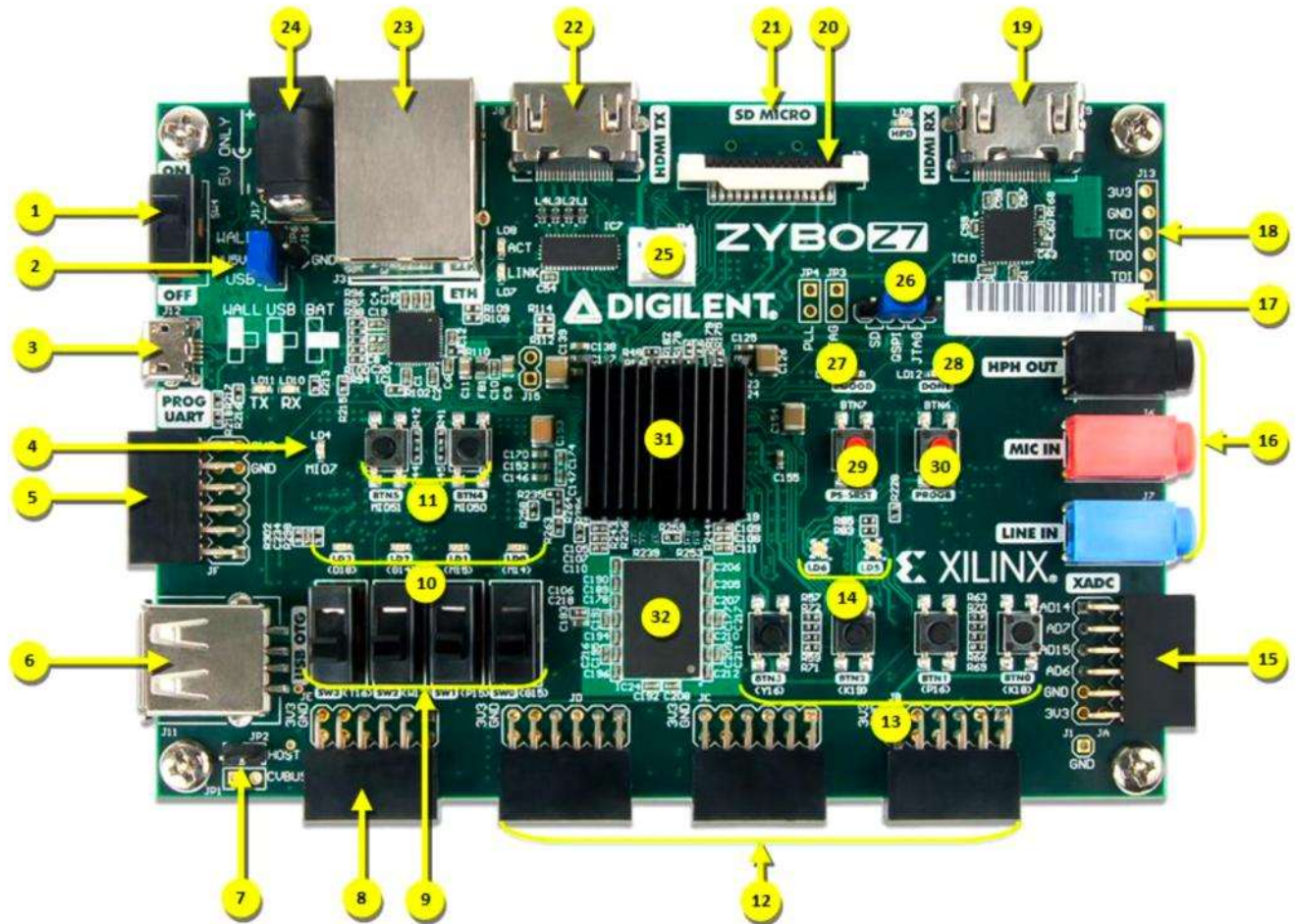
- Pcam camera connector with MIPI CSI-2 support
- HDMI sink port (input) with/without* CEC
- HDMI source port (output) with CEC
- Audio codec with stereo headphone, stereo line-in, and microphone jacks

Switches, Pushbuttons, & LEDs

- 6 push-buttons (2 processor connected)
- 4 slide switches
- 5 LEDs (1 processor connected)
- 2 RGB LEDs (1*)

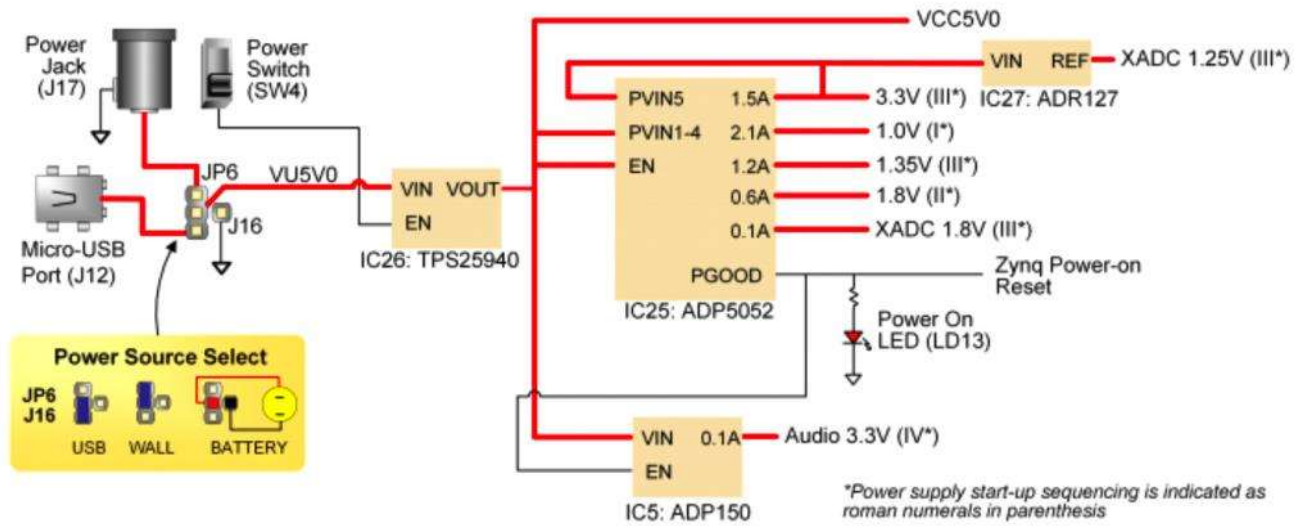
Expansion Connectors

- 6 Pmod ports (5*)
 - 8 Total Processor I/O
 - 40 Total FPGA I/O (32*)
 - 4 Analog capable 0-1.0V differential pairs to XADC



Callout	Description	Callout	Description
1	Power Switch	17	Unique MAC address label
2	Power select jumper	18	External JTAG port
3	USB JTAG/UART port	19	HDMI input port
4	MIO User LED	20	Pcam MIPI CSI-2 port
5	MIO Pmod port	21	microSD connector (other side)
6	USB 2.0 Host/OTG port	22	HDMI output port
7	USB Host power enable jumper	23	Ethernet port
8	Standard Pmod port	24	External power supply connector
9	User switches	25	Fan connector (5V, three-wire) *
10	User LEDs	26	Programming mode select jumper
11	MIO User buttons	27	Power supply good LED
12	High-speed Pmod ports *	28	FPGA programming done LED
13	User buttons	29	Processor reset button
14	User RGB LEDs *	30	FPGA clear configuration button
15	XADC Pmod port	31	Zynq-7000
16	Audio codec ports	32	DDR3L Memory

- Power



b. Board Configuration (Booting)

2022년 11월 18일 금요일 오후 8:45

- Booting process
 - ▶ Stage 0 : reset & Boot Rom 실행
copy FSBL from non-volatile to OCM >> execute **FSBL**
 - ▶ Stage 1 : ps configuration (DDR etc.)
pl configuration (**bitstream**)
load user **application** >> execution hand off
 - ▶ Stage 2 : execution of User application
simple baremetal or Linux applications
- Boot Image : FSBL + bitstream + application
 - Xilinx SDK를 이용하여 생성
 - Store into microSD, Quad SPI Flash (EEROM)
- Bootmode (configuration mode)

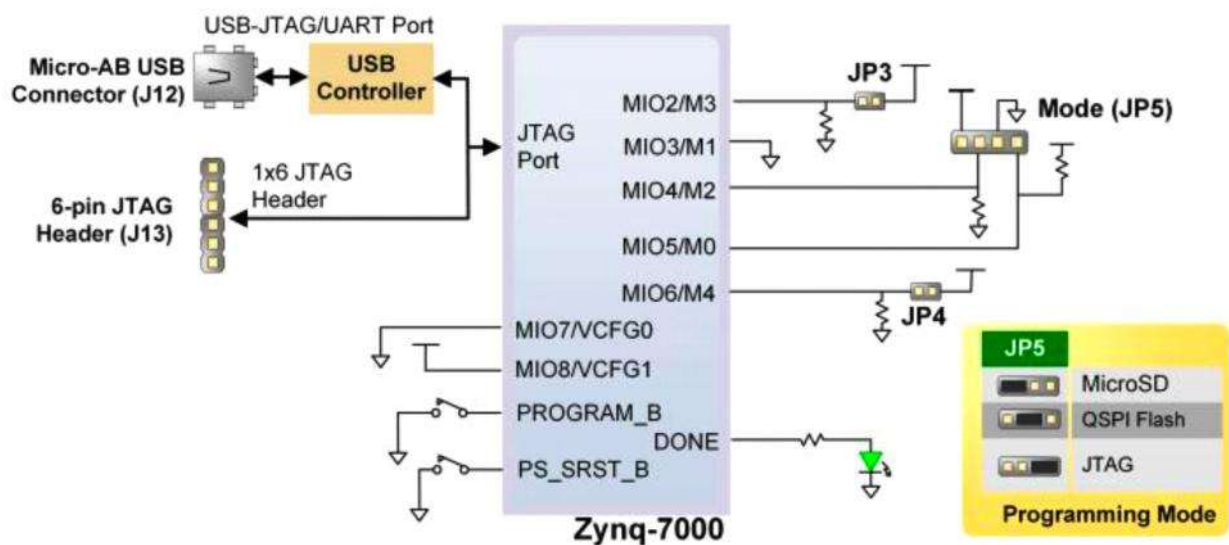


Figure 2.1. Zybo Z7 configuration pins.

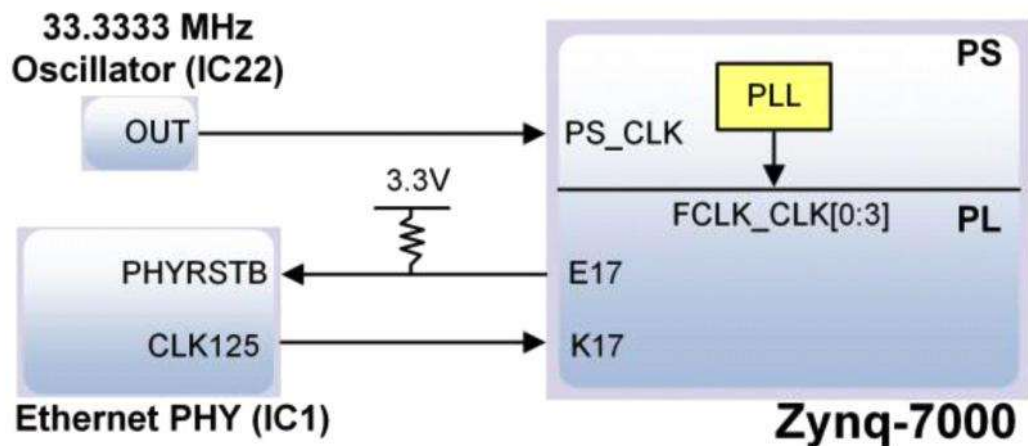
- **microSD bootmode**
- **QSPI bootmode**
- **JTAG mode**

c. clocks and IOs

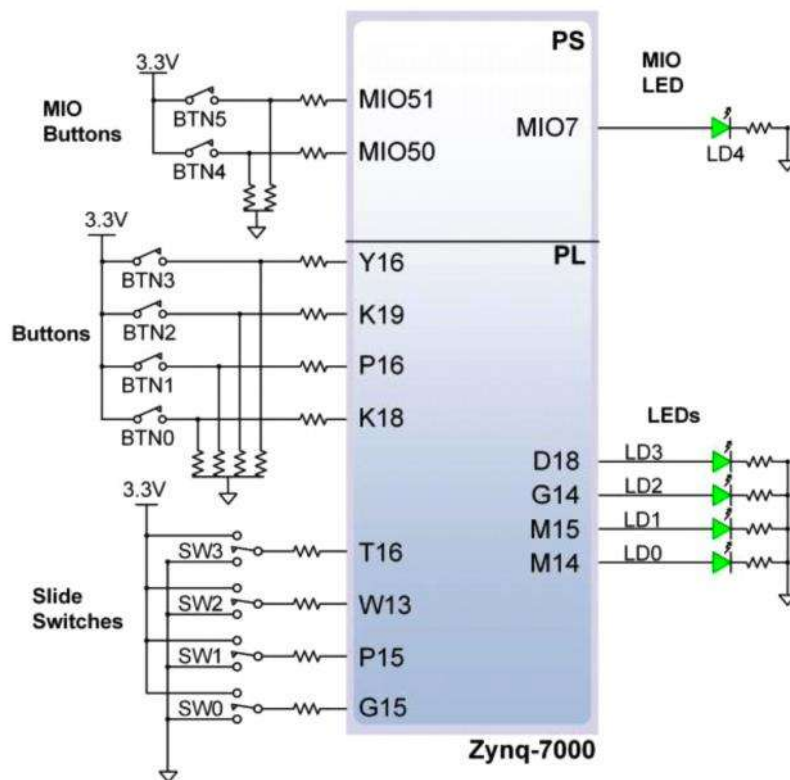
2022년 11월 18일 금요일 오후 9:01

- Clocks

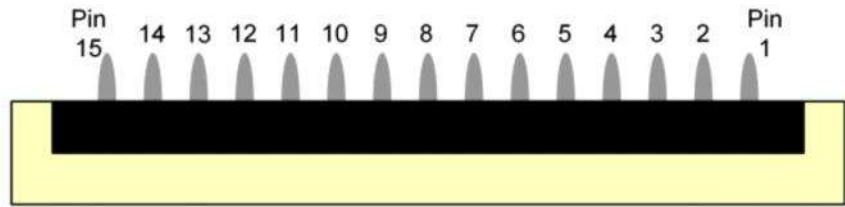
Figure 5.1 outlines the clocking scheme used on the Zybo Z7. Note that the reference clock output from the Ethernet PHY is used as the 125 MHz reference clock to the PL, in order to cut the cost of including a dedicated oscillator for this purpose. Keep in mind that CLK125 will be disabled when the Ethernet PHY is held in hardware reset by driving the PHYRSTB signal low.



- Basic IOs



- PCAM port

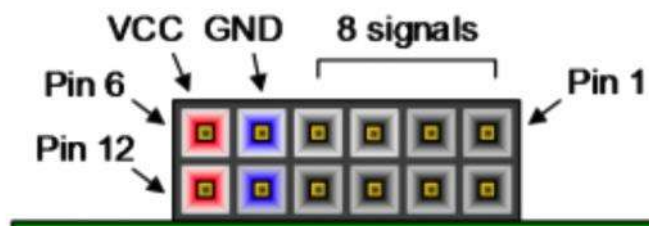


**Pin 1 will be indicated with a small white circle on the PCB*

Pin Number	Function	Zybo Z7 Connection Details
1	GND	GND
2	MIPI CSI-2 Lane 0 (-)	Terminated and connected to 2 FPGA pins as described in XAPP894
3	MIPI CSI-2 Lane 0 (+)	Terminated and connected to 2 FPGA pins as described in XAPP894
4	GND	GND
5	MIPI CSI-2 Lane 1 (-)	Terminated and connected to 2 FPGA pins as described in XAPP894
6	MIPI CSI-2 Lane 1 (+)	Terminated and connected to 2 FPGA pins as described in XAPP894
7	GND	GND
8	MIPI CSI-2 Clock (-)	Terminated and connected to 2 FPGA pins as described in XAPP894
9	MIPI CSI-2 Clock (+)	Terminated and connected to 2 FPGA pins as described in XAPP894
10	GND	GND
11	GPIO	Direct Connection to FPGA
12	GPIO	Direct Connection to FPGA
13	SCL	Direct Connection to FPGA, with 1.5 KOhm Pull-up
14	SDA	Direct Connection to FPGA, with 1.5 KOhm Pull-up
15	3V3	3.3 V Power rail

Table 15.1. Pcam Pin-out.

- PMOD Ports
 - 2×6, right-angle, 100-mil spaced female connectors, standard 2×6 pin headers
 - Provides
 - two 3.3V VCC signals (pins 6 and 12)
 - two Ground signals (pins 5 and 11)
 - eight logic signals



- Zybo has 6 PMOD

	Pmod JA	Pmod JB *	Pmod JC	Pmod JD	Pmod JE	Pmod JF
Pmod Type	XADC	High-Speed	High-Speed	High-Speed	Standard	MIO
Pin 1	N15	V8	V15	T14	V12	MIO-13
Pin 2	L14	W8	W15	T15	W16	MIO-10
Pin 3	K16	U7	T11	P14	J15	MIO-11
Pin 4	K14	V7	T10	R14	H15	MIO-12
Pin 7	N16	Y7	W14	U14	V13	MIO-0
Pin 8	L15	Y6	Y14	U15	U17	MIO-9
Pin 9	J16	V6	T12	V17	T17	MIO-14
Pin 10	J14	W6	U12	V18	Y17	MIO-15

- Tnadard Pmod: connected to the **Zynq PL** via 200 Ohm series resistors.
- MIO Pmod : connected to the MIO bus in the **Zynq PS** via 200 Ohm series resistors
- XADC : wired to the auxiliary analog input pins of the PL
- High-Speed Pmod : data signals routed as impedance matched differential pairs for maximum switching speeds (pins 1 and 2, pins 3 and 4, pins 7 and 8, and pins 9 and 10)